
Introdúcere in JDBC

Ioana Ciuciu

ioana.ciuciu@ubbcluj.ro

<http://www.cs.ubbcluj.ro/~oana/>

Planificare

Saptama na	Curs	Seminar	Laborator
S1	1. Concepte fundamentale ale bazelor de date. Modelare conceptuala	1. Modelul Entitate-Relatie. Modelul relational	1. Modelarea unei BD in modelul ER si implementarea ei in SQL Server
S2	2. Modelul relational de organizare a bazelor de date. Modelare conceptuala		
S3	3. Gestiunea bazelor de date relationale cu limbajul SQL (DDL)	2. Limbajul SQL – definirea si actualizarea datelor	2. Interogari SQL
S4	4. Gestiunea bazelor de date relationale cu limbajul SQL (DML)		
S5-6	5-6. Dependente functionale, forme normale	3. Limbajul SQL – regasirea datelor	3. Interogari SQL avansate
S7	7. JDBC (Java Database Connectivity)	4. Proceduri stocate	4. Proceduri stocate. View. Trigger
S8	8. Interogarea bazelor de date relationale cu operatori din algebra relationala		
S9	9. Structura fizica a bazelor de date relationale	5. View-uri. Functii definite de utilizator. Trigger	
S10-11	10-11. Indeksi. Arbori B. Fisiere cu acces direct	6. Formele normale ale unei relatii. Indeksi	
S12	12. Evaluarea interogarilor in bazele de date relationale		
S13	13. Extensii ale modelului relational si baze de date NoSQL	7. Probleme	Examen practic
S14	14. Aplicatii		

Planul cursului

- ▶ Curs 7 – Java Database Connectivity (JDBC):
 - ▶ Obiectivele JDBC
 - ▶ Arhitectura JDBC
 - ▶ Utilizarea JDBC
 - ▶ Pachetul java.sql
 - ▶ Relatia dintre tipuri SQL si tipuri Java
 - ▶ Interogari SQL cu JDBC
 - ▶ Instructiuni preparate
 - ▶ Referinte bibliografice



Obiectivele JDBC

- ▶ JDBC: Java Database Connectivity
- ▶ JDBC permite executia interogarilor SQL asupra unei baze de date pornind de la un program Java



Ce este JDBC (Java Database Connectivity)

▶ API JDBC

- ▶ API (Application Programming Interface) JDBC permite executia de interogari SQL
- ▶ API JDBC apartine JDK (Java Development Kit), prin intermediul pachetului `java.sql`

▶ Driver JDBC

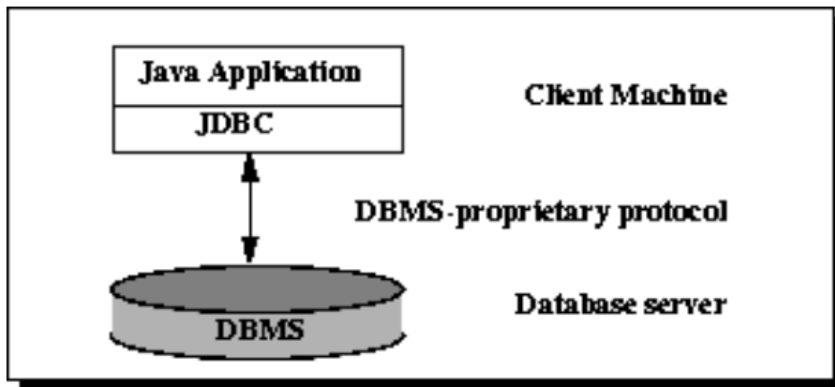
- ▶ Driver-ul JDBC stabileste o conexiune retea cu baza de date, pentru comunicarea cu aceasta
- ▶ Exista drivere JDBC pentru diferite baze de date (SQL Server, MySQL, PostgreSQL, Oracle, etc.)



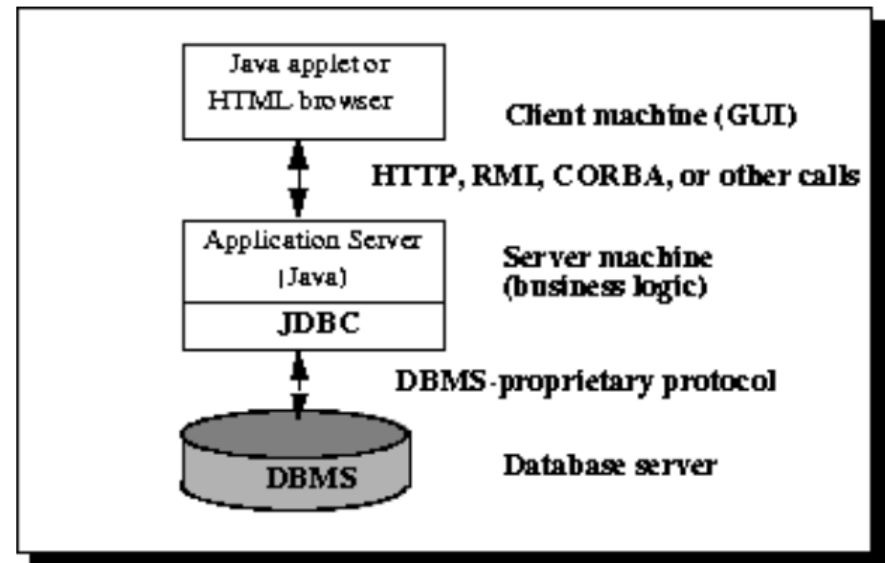
Arhitectura JDBC

- ▶ API-ul JDBC suporta atat arhitectura pe doua nivele, cat si arhitectura pe trei nivele pentru accesul la baza de date

Arhitectura pe doua nivele pentru accesul la date



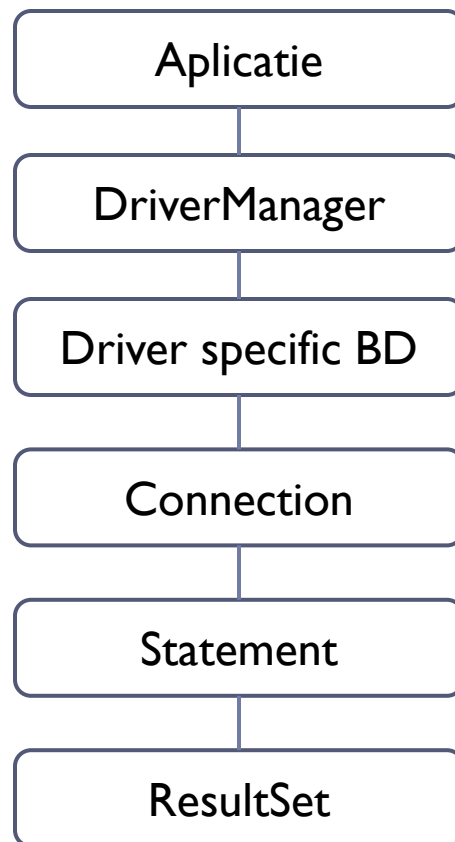
Arhitectura pe trei nivele pentru accesul la date



Sursa: <https://docs.oracle.com/javase/tutorial/jdbc/overview/architecture.html>

Arhitectura JDBC

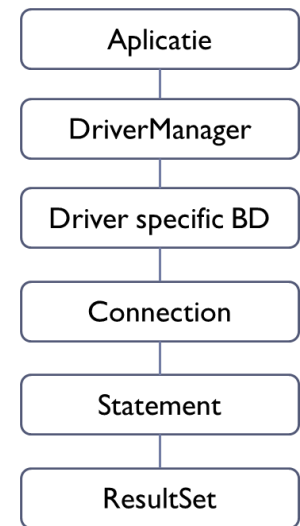
- ▶ Clasele si interfetele pentru JDBC se gasesc in pachetul java.sql
- ▶ O aplicatie ce utilizeaza JDBC foloseste urmatoarele clase si interfete:



Arhitectura JDBC

- In aplicatii, comunicarea cu o baza de date necesita urmatoorii pasi:

1. Se apeleaza la DriverManager cerand un driver specific pentru baza de date
2. Driverul specific creeaza legatura cu baza de date si returneaza un obiect de tip Connection
3. Cu ajutorul obiectului de tip Connection se creeaza un obiect Statement care contine si o cerere SQL catre baza de date
4. Obiectul Statement returneaza rezultatele intr-un obiect ResultSet



Utilizarea JDBC - Etape

► Adaugarea driver-ului JDBC la proiectul Eclipse

1. Descarcati driverul jdbc (fisier .jar): [Download - JDBC Driver for SQL Server | Microsoft Learn](#)
2. Click dreapta pe numele proiectului din Eclipse
3. Selectati Build Path → Add External Archives...
4. Selectati fisierul descarcat mai sus (ex., mssql-jdbc-7.4.1.jre8 sau mai recent)
5. Click pe Open
6. Driverul este acum adaugat proiectului vostru



Utilizarea JDBC - Etape

```
import java.sql.*; //API JDBC

public class ExempluJDBC {

    //program principal

        //incarcarea driverului

        //deschiderea conexiunii

        //executia unei interogari SQL

}
```



Utilizarea JDBC – incarcarea driverului si realizarea conexiunii

```
import java.sql.*;

public class ExempluJDBC {
    public static void main(String[] args) {
        String connectionUrl =
            "jdbc:sqlserver://yourserver:1433;"
                + "instanceName=yourinstance;"
                + "database=databasename;"
                + "integratedSecurity=true;";
        try (Connection connection =
            DriverManager.getConnection(connectionUrl);) {
            //cod
        }
        catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```



Utilizarea JDBC – executia unei interogari SQL

```
import java.sql.*;

public class ExempluJDBC {
    public static void main(String[] args) {
        //formarea connectionURL (vezi slide precedent)
        ResultSet rs = null;
        try (Connection connection =
DriverManager.getConnection(connectionUrl);
Statement stmt = connection.createStatement();) {
            //crearea si executia instructiunii SQL
            String selectSql = "SELECT * from Persoane";
            rs = stmt.executeQuery(selectSql);

            //prelucrarea si afisarea rezultatelor
            while(rs.next()){
                System.out.println(rs.getString(1)+ " "
+ rs.getString(2));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Pachetul java.sql

- ▶ **Principalele clase si interfete ale pachetului java.sql**
 - ▶ **Connection**
 - ▶ Conexiunea la o baza de date
 - ▶ **Driver**
 - ▶ Interfata cu driverul JDBC al bazei de date
 - ▶ **ResultSet**
 - ▶ O tabela reprezentand date din baza de date
 - ▶ **Statement**
 - ▶ Obiect folosit la executia unei interogari SQL
 - ▶ **SQLException**
 - ▶ Eroare in timpul accesului la baza de date
 - ▶ **SQLWarning**
 - ▶ Mesaj de avertizare in timpul accesului la baza de date



Interfata java.sql.ResultSet

- ▶ **ResultSet:**
 - ▶ Tabela reprezentand date din baza de date
- ▶ **Accesul la valorile unui obiect ResultSet**
 - ▶ Type getType(int numarulColoanei)
 - ▶ Exemple:
 - ▶ String getString(int columnIndex);
 - ▶ int getInt(int columnIndex);
 - ▶ float getFloat(int columnIndex);
 - ▶ Type getType(String numeleColoanei)
 - ▶ Exemple:
 - ▶ String getString(String columnLabel);
 - ▶ int getInt(String columnLabel);
 - ▶ float getFloat(String columnLabel);



Exceptii SQL

- ▶ Clasa `SQLException` din pachetul `java.sql`
- ▶ Informatii continute de catre o instanta `SQLException`
 - ▶ Descrierea erorii:
 - ▶ Metoda `SQLException.getMessage()`
 - ▶ Codul starii SQL:
 - ▶ Coduri standardizate de catre ISO/ANSI si Open Group (X/Open)
 - ▶ Metoda `SQLException.getSQLState()`
 - ▶ Codul erorii:
 - ▶ Coduri specifice implementarii SGBD
 - ▶ Metoda `SQLException.getErrorCode()`
 - ▶ Cauza erorii:
 - ▶ Unul sau mai multe obiecte `Throwable` ca si cauza a `SQLException`
 - ▶ Metoda `SQLException.getCause()`



Mesaje de avertizare SQL

- ▶ Clasa `SQLWarning` din pachetul `java.sql`
 - ▶ Sub-clasa a clasei `SQLException`
- ▶ Cand se produc?
 - ▶ Mesaj de avertizare provenind de la **accesul** la BD
 - ▶ Nu opreste executia aplicatiei (spre deosebire de exceptii)
 - ▶ Mesaj **asociat unui obiect Connection, obiect Statement sau unui obiect ResultSet**
- ▶ Exemplu:
 - ▶ Mesaj de avertizare catre un utilizator anuntand ca un privilegiu pe care a incercat sa il revoce nu a fost revocat
- ▶ Informatii asupra `SQLWarning`:
 - ▶ Recuperarea mesajelor de avertizare:
 - ▶ Metoda `Connection.getWarnings()`
 - ▶ Metoda `Statement.getWarnings()`
 - ▶ Metoda `ResultSet.getWarnings()`
 - ▶ Urmatorul mesaj de avertizare:
 - ▶ Metoda `SQLWarning.getNextMessage()`



Relatia dintre tipuri SQL si tipuri Java

Tip SQL	Tip Java	Metoda
CHAR	String	getString()
VARCHAR	String	getString()
LONGVARCHAR	String	getString()
NUMERIC	java.math.BigDecimal	getBigDecimal()
DECIMAL	java.math.BigDecimal	getBigDecimal()
BIT	boolean	getBoolean()
TINYINT	byte	getByte()
SMALLINT	short	getShort()
INTEGER	int	getInt()
BIGINT	long	getLong()
REAL	float	getFloat()
DOUBLE	double	getDouble()
BINARY	byte[]	getBytes()
VARBINARY	byte[]	getBytes()
LONGVARBINARY	byte[]	getBytes()
DATE	java.sql.Date	getDate()
TIME	java.sql.Time	getTime()
TIMESTAMP	java.sql.Timestamp	getTimestamp()

Interogari SQL cu JDBC

- ▶ DML (Data Manipulation Language)

- ▶ SELECT

- ▶ Consultarea datelor dintr-o BD

- ▶ Exemplu:

```
Statement st = conn.createStatement();  
ResultSet rs = st.executeQuery("SELECT nume, prenume FROM Persoane");  
st.close();
```



Interogari SQL cu JDBC

► DML (Data Manipulation Language)

► INSERT

► Adaugarea de noi inregistrari intr-o BD

► Exemplu:

```
Statement st = conn.createStatement();  
int nr = st.executeUpdate("INSERT INTO  
Persoane(id,nume,prenume,localitate) VALUES  
(15,'Stefanescu', 'Andra', 'Cluj-Napoca')");  
System.out.println("Numar de inregistrari inserate = "+nr);  
st.close();
```



Interogari SQL cu JDBC

► DML (Data Manipulation Language)

► UPDATE

► Modificarea datelor intr-o tabela a unei BD

► Exemplu:

```
Statement st = conn.createStatement();  
int nr = st.executeUpdate("UPDATE Persoane SET Localitate = 'Sibiu'  
WHERE nume = 'Stefanescu'");  
System.out.println("Numar de inregistrari actualizate = "+nr);  
st.close();
```



Interogari SQL cu JDBC

► DML (Data Manipulation Language)

► DELETE

- Stergerea inregistrarilor dintr-o tabela a unei BD

► Exemplu:

```
Statement st = conn.createStatement();  
int nr = st.executeUpdate("DELETE from Persoane WHERE nume =  
'Stefanescu'");  
System.out.println("Numar de inregistrari sterse = "+nr);  
st.close();
```



Manipularea unui obiect ResultSet

▶ `java.sql.ResultSet`

- ▶ Metoda `ResultSet.first()`
 - ▶ Pozitioneaza cursorul pe prima inregistrare a tabeli returnate de `ResultSet`
- ▶ Metoda `ResultSet.next()`
 - ▶ Pozitioneaza cursorul pe urmatoarea inregistrare a tabeli returnate de `ResultSet`
- ▶ Metoda `ResultSet.prvious()`
 - ▶ Pozitioneaza cursorul pe inregistrarea anterioara a tabeli returnate de `ResultSet`
- ▶ Metoda `ResultSet.beforeFirst()`
 - ▶ Pozitioneaza cursorul inainte de prima inregistrare a tabeli returnate de `ResultSet`
- ▶ Metoda `ResultSet.afterLast()`
 - ▶ Pozitioneaza cursorul dupa ultima inregistrare a tabeli returnate de `ResultSet`
- ▶ Metoda `ResultSet.absolute(int row)`
 - ▶ Pozitioneaza cursorul pe o inregistrare data a tabeli returnate de `ResultSet`
- ▶ Metoda `ResultSet.relative(int row)`
 - ▶ Muta cursorul inainte/inapoi cu un anumit numar de pozitii in tabela returnata de `ResultSet`



Recuperarea valorilor rezultate dintr-o interogare SQL

► Exemplu:

```
Statement st = conn.createStatement();
```

```
ResultSet rs = st.executeQuery("SELECT nume,  
prenume, localitate FROM Persoane");
```

```
while (rs.next()) {  
    String nume = rs.getString(1);  
    String prenume = rs.getString(2);  
    String localitate = rs.getString(3);  
    System.out.println(nume + "\t" + prenume +  
"\t" + localitate);  
}
```

```
st.close();
```



Interogari SQL cu JDBC

- ▶ DDL (Data Definition Language)

- ▶ CREATE TABLE

- ▶ Exemplu:

```
Statement st = conn.createStatement();  
st.executeUpdate("CREATE TABLE Persoane( int  
id PRIMARY KEY, nume VARCHAR(30), prenume  
VARCHAR(30), localitate VARCHAR(50)");  
st.close();
```



Interogari SQL cu JDBC

- ▶ DDL (Data Definition Language)
 - ▶ DROP TABLE
 - ▶ Exemplu:

```
Statement st = conn.createStatement();  
st.executeUpdate("DROP TABLE Persoane");  
st.close();
```



Interogari SQL cu JDBC

- ▶ DDL (Data Definition Language)

- ▶ ALTER TABLE

- ▶ Exemplu:

```
Statement st = conn.createStatement();  
st.executeUpdate("ALTER TABLE Persoane ADD  
varsta int");  
st.executeUpdate("ALTER TABLE Persoane DROP  
COLUMN localitate");  
st.close();
```



Instructiuni preparate

- ▶ Frecvent, **aplicatiile** care lucreaza cu BD folosesc **acelasi cod SQL de mai multe ori (reutilizarea codului)**
- ▶ SGBD faciliteaza crearea **instructiunilor preparate** prin intermediul clasei **PreparedStatement** – subclasa a clasei **Statement**
- ▶ Clasa **PreparedStatement** are un **constructor** care primeste ca parametru un string reprezentand instructiunea SQL:

```
PreparedStatement pstmt = conn.prepareStatement("SELECT  
nume FROM Persoane WHERE localitate=?");
```

- ▶ **Instructiunile preparate** – trebuie **create, analizate si optimizate o singura data** in BD, dupa care pot fi **utilizate** in aplicatii **de cate ori e nevoie**
-



Instructiuni preparate

- ▶ Clasa PreparedStatement are un **constructor** care primeste ca parametru un string reprezentand instructiunea SQL:

```
PreparedStatement pstmt = conn.prepareStatement("SELECT  
nume FROM Persoane WHERE localitate=?");
```



- ▶ Se poate observa ca instructiunea SQL contine si **semne de intrebare**
- ▶ Semnele de intrebare din instructiunea SQL reprezinta **paramentrii interogarii**
- ▶ Acesti parametri pot fi setati inainte de executia propriu-zisa a codului



Instructiuni preparate

► Exemplu (fara instructiuni preparate):

```
public static void modifLocalitate(String[] nume, String[] localitate) throws SQLException
{
    try{
        // se creeaza o noua conexiune si un nou statement
        Connection conn = new Connection();
        Statement st = conn.createStatement();

        int lungime = nume.length;
        if (localitate.length != lungime) {//eroare}

        for (int i = 0; i < lungime; i++) {
            int nr = st.executeUpdate("UPDATE Persoane SET localitate = " + localitate[i]
                                     + "WHERE nume = '" + nume[i] + "'");
        }
    } finally {...}
}
```

Inconvenient: solutie costisitoare

In acest exemplu pentru fiecare nume de persoana furnizat ca si parametru s-a executat cate o instructiune SQL, timpul de executie astfel devenind foarte mare



Instructiuni preparate

► Exemplu (cu instructiuni preparate):

```
public static void modifLocalitate(String[] nume, String[] localitate) throws SQLException
{
    // se creeaza o noua conexiune si un nou prepared statement
    // formarea connectionString (vezi slide precedent)
    String sql1 = "UPDATE Persoane SET localitate = ? WHERE nume = ?";
    try (Connection con=DriverManager.getConnection(connUrl);
         PreparedStatement pst = con.prepareStatement(sql1);) {
        System.out.println("Connection!");

        int lungime = nume.length;
        if (localitate.length != lungime) {
            System.out.println("Numar incorect de parametri");
        }
        for (int i = 0; i < lungime; i++) {
            pst.setString(1, localitate[i]);
            pst.setString(2,nume[i]);
            int nr = pst.execute();
        }
    } finally {...}
}
```

Avantaj:

In acest exemplu, codul SQL se compileaza o singura data pe partea de baza de date si la fiecare apel **execute()** se va executa acel cod precompilat cu parametrii inlocuiti. Parametrii interogarii sunt setati de catre metodele *setXXX()* ale clasei *PreparedStatement*. Primul parametru al acestor metode este numarul de ordine al parametrului din secventa SQL care se inlocuieste, iar al doilea parametru este valoarea cu care se inlocuieste-----

Informatii asupra bazei de date

▶ `java.sql.MetaData`

- ▶ Meta-informatii asupra bazei de date
- ▶ Prin apelarea metodei `Connection.getMetaData()`
- ▶ Informatii disponibile:
 - ▶ Tabele existente in BD
 - ▶ Numele utilizatorului
 - ▶ Numele driver-ului JDBC
 - ▶ Versiunea driver-ului JDBC
 - ▶ Etc,



Informatii asupra ResultSet

- ▶ **java.sql.ResultSetMetaData**
 - ▶ Meta-informatii asupra rezultatului unei interogari SQL
 - ▶ Prin apelarea metodei `ResultSet.getMetaData()`
 - ▶ Prin apelarea metodei `PreparedStatement.getMetaData()`
- ▶ Informatii disponibile:
 - ▶ Numarul de coloane din tabela rezultat
 - ▶ Numele tabelelor initiale (pe baza carora s-a obtinut rezultatul)
 - ▶ Tipurile asociate coloanelor tabelului rezultat
 - ▶ Etc.



Informatii asupra parametrilor

- ▶ **Java.sql.ParameterMetaData**
 - ▶ Meta-informatii asupra parametrilor utilizati impreuna cu un obiect `PreparedStatement`
 - ▶ Prin apelarea metodei `PreparedStatement.getParameterMetaData()`
- ▶ Informatii disponibile:
 - ▶ Numarul de parametri ai unui obiect `PreparedStatement`
 - ▶ Tipurile SQL ai parametrilor unui obiect `PreparedStatement`
 - ▶ Numele claselor Java ai parametrilor unui obiect `PreparedStatement`
 - ▶ Etc.



Utilitatea meta-informatiilor

- ▶ **Avantaje ale utilizarii API-ului JDBC pentru accesul la meta-informatii intr-un program Java:**
 - ▶ Parte de cod independenta de interogarea SQL
 - ▶ Cod generic, refolosibil in mai multe aplicatii



Referinte

- ▶ Date, C.J., An Introduction to Database Systems (8th Edition), Addison-Wesley, 2004
- ▶ Date, C.J., A Guide to the SQL Standard, Addison Wesley
- ▶ Date, C.J., A Guide to DB2, Addison Wesley
- ▶ Garcia-Molina, H., Ullman, J., Widom, J., *Database Systems: The Complete Book*, Pearson Prentice Hall, 2008
- ▶ Ramakrishnan, R., Database Management Systems. McGraw-Hill, 2007
- ▶ Curs bazat partial pe urmatoarele resurse:
 - ▶ Bouchenak, S., UJF Grenoble, Curs de baze de date, JDBC, Universitatea Joseph Fourier Grenoble
 - ▶ https://ms.sapientia.ro/~manyi/teaching/oop/oop_romanian/curs10/curs10.html

